

UNIVERSIDAD TÉCNICA NACIONAL

Carrera de Ingeniería del Software

ISW-1013: Calidad del Software

ANÁLISIS Y CORRECCIÓN DE VULNERABILIDADES SQL INJECTION EN APLICACIÓN WEB REAL

INTEGRANTES DEL EQUIPO	
Carlos Andrés Vargas Berrocal	208280648
Jefry Gerardo Jiménez Rocha	208320789
Karla Melissa Madriz	207900020

Fecha: 24 de March de 2026

Repositorio: https://github.com/Aigen1111/sqli_utn.git

V-01: SQL INJECTION EN LOGIN

DESCRIPCIÓN DEL PROBLEMA

La función de login construye la consulta SQL concatenando directamente los valores del usuario sin validación. Esto permite que un atacante inyecte código SQL arbitrario para modificar la lógica de la consulta.

CÓDIGO VULNERABLE (Línea 166)

query = f"SELECT id, username, role FROM users WHERE username = '{username}' AND password = '{password}'"

PAYLOADS EXPLORADOS

Payload	Resultado
admin' --	Bypassed: Entra como admin sin contraseña
' OR '1'='1' --	Bypassed: Acceso como primer usuario
admin' OR '1'='1	Bypassed: Acceso sin comentario SQL

EVIDENCIA: Ataque exitoso con admin' --

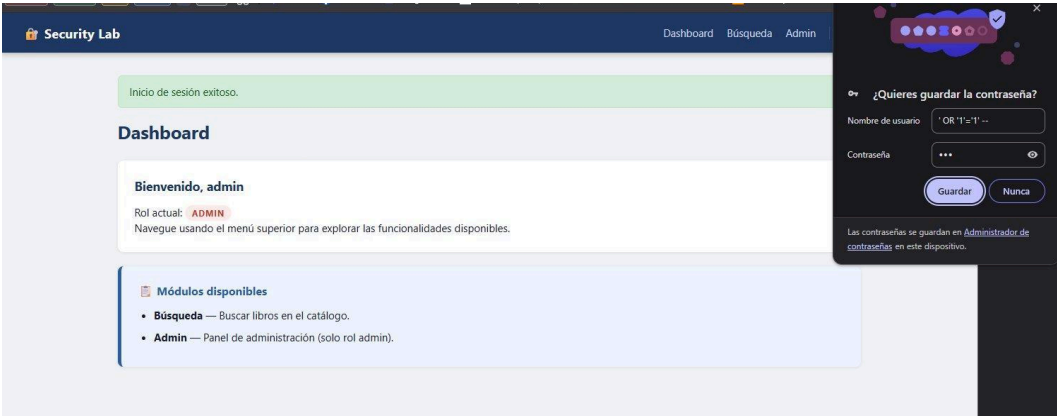


Figura 1: Usuario logeado como admin usando payload admin' --

EVIDENCIA: Ataque exitoso con ' OR '1'='1' --

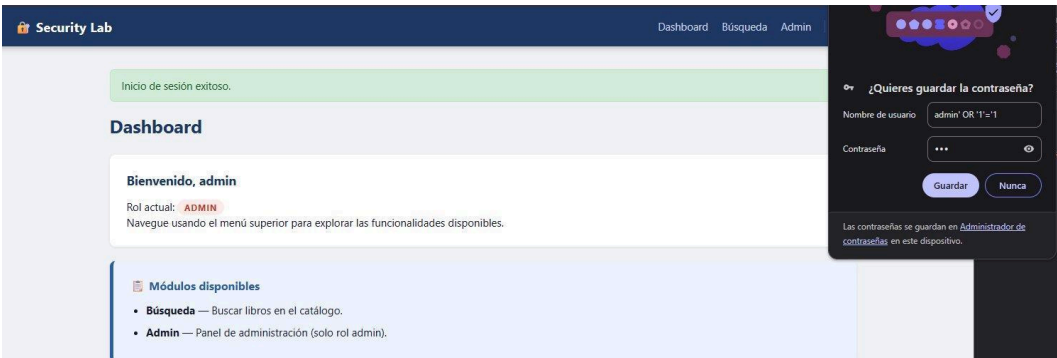


Figura 2: Acceso como admin usando OR condition

¿POR QUÉ ES PELIGROSA?

Impacto CRÍTICO: Un atacante puede acceder a cualquier cuenta sin conocer la contraseña. Esto permite escalado de privilegios a nivel administrador.

CORRECCIÓN IMPLEMENTADA

Usar consultas parametrizadas donde los valores se pasan separados de la estructura SQL:

```
query = "SELECT id, username, role FROM users WHERE username = ?"  
user = conn.execute(query, (username,)).fetchone()
```

V-02: SQL INJECTION EN BÚSQUEDA (UNION-BASED)

DESCRIPCIÓN DEL PROBLEMA

La función de búsqueda concatena el término directamente. Un atacante puede usar UNION SELECT para extraer datos de otras tablas, incluidas todas las contraseñas.

CÓDIGO VULNERABLE (Línea 230)

```
raw_query = f"SELECT id, title, author, category FROM books WHERE title LIKE '%{term}%' OR author LIKE '%{term}%' OR category LIKE '%{term}%'"
```

PAYLOADS EXPLORADOS

Payload	Resultado
%' UNION SELECT 1,2,3,4 --	Verifica estructura: 4 columnas
%' UNION SELECT id, username, username, role FROM users --	Extrae: admin, analyst, student
%' UNION SELECT id, username, password, role FROM users --	Extrae TODAS las contraseñas en TEXTO PLANO

EVIDENCIA: Verificación de número de columnas (UNION SELECT 1,2,3,4)

Búsqueda de Libros

Término de búsqueda

Buscar por título, autor o categoría...

Buscar

⚠ Consulta SQL ejecutada (esto no debería mostrarse)

SELECT id, title, author, category FROM books WHERE title LIKE '%%' UNION SELECT 1,2,3,4 --%' OR author LIKE '%%' UNION SELECT 1,2,3,4 --%' OR category LIKE '%%' UNION SELECT 1,2,3,4 --%'

Resultados (9 encontrados)

ID	Título	Autor	Categoría
1	2	3	4
1	Secure Coding Fundamentals	J. Howard	security
2	Flask Web Patterns	A. Miller	development
3	Practical SQL	A. DeBarros	database
4	Threat Modeling Essentials	M. Silva	security
5	Performance Testing Handbook	R. Jones	qa
6	OWASP Testing Guide	OWASP Team	security
7	Clean Code	R. Martin	development
8	The Web Application Hacker	Stuttard	security

Figura 3: Payload UNION SELECT 1,2,3,4 retorna 4 columnas

EVIDENCIA: Extrayendo nombres de usuarios

Búsqueda de Libros

Término de búsqueda

Buscar

⚠ Consulta SQL ejecutada (esto no debería mostrarse)

```
SELECT id, title, author, category FROM books WHERE title LIKE '%%' UNION SELECT id, username, username, role FROM users --%' OR author LIKE '%%' UNION SELECT id, username, username, role FROM users --%' OR category LIKE '%%' UNION SELECT id, username, username, role FROM users --%'
```

Resultados (11 encontrados)

ID	Título	Autor	Categoría
1	Secure Coding Fundamentals	J. Howard	security
1	admin	admin	admin
2	Flask Web Patterns	A. Miller	development
2	analyst	analyst	user
3	Practical SQL	A. DeBarros	database
3	student	student	user
4	Threat Modeling Essentials	M. Silva	security
5	Performance Testing Handbook	R. Jones	qa
6	OWASP Testing Guide	OWASP Team	security
7	Clean Code	R. Martin	development
8	The Web Application Hacker	Stuttard	security

Figura 4: UNION SELECT extrae admin, analyst, student

EVIDENCIA CRÍTICA: Extrayendo TODAS las contraseñas en TEXTO PLANO

Búsqueda de Libros

Término de búsqueda

Buscar

⚠ Consulta SQL ejecutada (esto no debería mostrarse)

```
SELECT id, title, author, category FROM books WHERE title LIKE '%%' UNION SELECT id, username, password, role FROM users --%' OR author LIKE '%%' UNION SELECT id, username, password, role FROM users --%' OR category LIKE '%%' UNION SE
```

Resultados (11 encontrados)

ID	Título	Autor	Categoría
1	Secure Coding Fundamentals	J. Howard	security
1	admin	Admin123	admin
2	Flask Web Patterns	A. Miller	development
2	analyst	Analyst123	user
3	Practical SQL	A. DeBarros	database
3	student	Student123	user
4	Threat Modeling Essentials	M. Silva	security
5	Performance Testing Handbook	R. Jones	qa
6	OWASP Testing Guide	OWASP Team	security
7	Clean Code	R. Martin	development
8	The Web Application Hacker	Stuttard	security

Figura 5: UNION SELECT extrae contraseñas: Admin123, Analyst123, Student123

¿POR QUÉ ES PELIGROSA?

Impacto CRÍTICO: Extrae TODAS las contraseñas de TODOS los usuarios en texto plano. Permite acceso a cualquier tabla de la base de datos.

CORRECCIÓN IMPLEMENTADA

```
query = "SELECT id, title, author, category FROM books WHERE title LIKE ? OR author LIKE ? OR category LIKE ?"
```

```
search_term = f"%{term}%"
```

```
books = conn.execute(query, (search_term, search_term, search_term)).fetchall()
```

V-03: CONTRASEÑAS EN TEXTO PLANO

DESCRIPCIÓN DEL PROBLEMA

Las contraseñas se almacenan sin cifrar. Se muestran en el panel Admin.

CONTRASEÑAS EXPUESTAS

Usuario	Contraseña (Texto Plano)
admin	Admin123
analyst	Analyst123
student	Student123

¿POR QUÉ ES PELIGROSA?

Impacto ALTO: Si la BD se compromete, el atacante tiene todas las contraseñas. Usuarios reutilizan contraseñas, permitiendo ataques en cascada.

CORRECCIÓN IMPLEMENTADA

```
from werkzeug.security import generate_password_hash, check_password_hash
```

```
users = [  
    ("admin", generate_password_hash("Admin123", method="pbkdf2"), "admin"),  
    ("analyst", generate_password_hash("Analyst123", method="pbkdf2"), "user"),  
]
```

```
if user and check_password_hash(user["password"], password):  
    # Contraseña correcta
```

V-04 y V-05: CONFIGURACIÓN INSEGURA

V-04: SECRET_KEY HARDCODEADA (Línea 37)

```
app.config["SECRET_KEY"] = "dev-secret-key-insegura-1234"
```

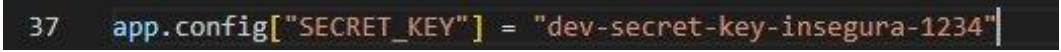
Problema: La clave está en el código y committed a Git. Un atacante puede forjar sesiones válidas.

Corrección: Cargar desde variable de entorno:

```
import os
```

```
app.config["SECRET_KEY"] = os.environ.get("SECRET_KEY", "fallback")
```

EVIDENCIA: SECRET_KEY en el código (Línea 37)



```
37 app.config["SECRET_KEY"] = "dev-secret-key-insegura-1234"
```

Figura 6: SECRET_KEY hardcodeada en app.py

V-05: debug=True ACTIVO (Línea 282)

```
app.run(host="0.0.0.0", port=5000, debug=True)
```

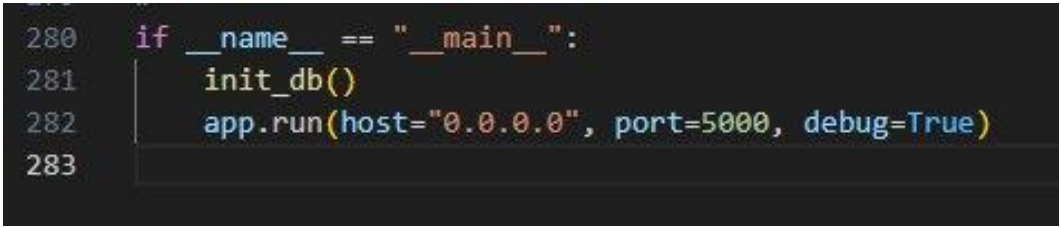
Problema: Modo debug expone un debugger interactivo en el navegador. RCE potencial.

Corrección:

```
debug_mode = os.environ.get("FLASK_ENV") == "development"
```

```
app.run(host="0.0.0.0", port=5000, debug=debug_mode)
```

EVIDENCIA: debug=True en el código (Línea 282)



```
280 if __name__ == "__main__":  
281     init_db()  
282     app.run(host="0.0.0.0", port=5000, debug=True)  
283
```

Figura 7: debug=True activo

V-06 y V-07: EXPOSICIÓN DE INFORMACIÓN

V-06: CONSULTA SQL EXPUESTA EN BÚSQUEDA

Problema: La página muestra la consulta SQL ejecutada, exponiendo la estructura de la BD.

Corrección: NO pasar raw_query al template. El usuario nunca debe ver detalles internos.

V-07: MENÚ ADMIN VISIBLE PARA TODOS

Problema: El enlace "Admin" aparece para todos. El backend lo bloquea, pero expone la opción.

EVIDENCIA: Menú Admin visible para usuario normal (analyst)

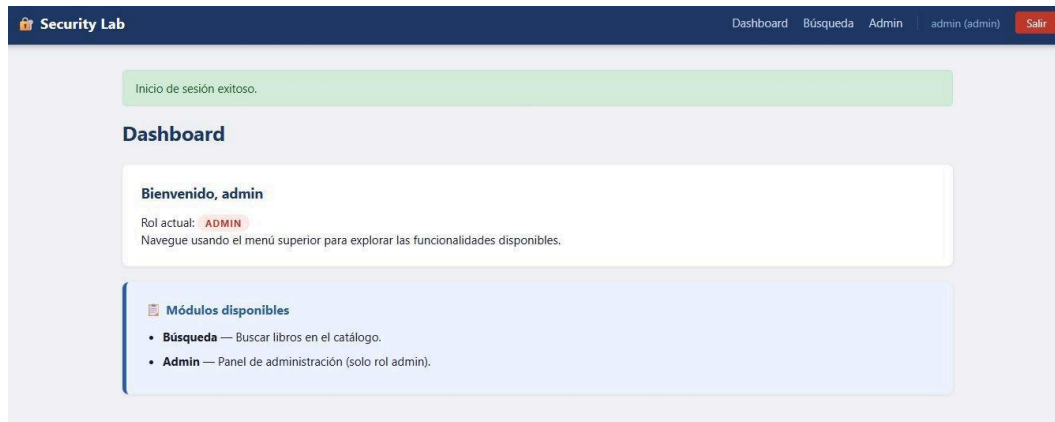


Figura 8: Usuario analyst ve el botón Admin en el menú (aunque no puede entrar)

Corrección: Mostrar solo si el usuario es admin:

```
{% if session.get("role") == "admin" %}
  <a href="{{ url_for("admin") }}">Admin</a>
{% endif %}
```

V-08: SIN PROTECCIÓN CSRF

DESCRIPCIÓN DEL PROBLEMA

Los formularios no tienen tokens CSRF. Un atacante puede forjar peticiones desde un sitio malicioso que se ejecutan con la identidad del usuario autenticado.

CORRECCIÓN IMPLEMENTADA

pip install Flask-WTF

```
from flask_wtf.csrf import CSRFProtect
csrf = CSRFProtect(app)
```

En templates:

```
<form method="post">
  {{ csrf_token() }}
  ...resto del formulario...
</form>
```

CONCLUSIONES Y LECCIONES APRENDIDAS

VULNERABILIDADES MÁS CRÍTICAS

V-01 (SQLi Login): Acceso como cualquier usuario sin contraseña

V-02 (SQLi Búsqueda): Extrae todas las contraseñas y datos sensibles

V-05 (debug=True): RCE - Ejecución remota de código

LECCIONES CLAVE

1. NUNCA concatenar input del usuario en SQL. Siempre usar consultas parametrizadas.
2. Las contraseñas DEBEN hashear. Usar bcrypt o Argon2, nunca SHA sin salt.
3. Los secretos NO van en el código. Usar variables de entorno.
4. Debug DEBE estar desactivado en producción. Es una shell interactiva.
5. NO exponer información interna. Nunca mostrar queries SQL, errores técnicos.
6. Control de acceso en UI Y backend. Esconder opciones que el usuario no puede usar.
7. Protección CSRF en TODOS los formularios. Tokens únicos por sesión.

CONCLUSIÓN FINAL

Esta actividad demostró que la seguridad en aplicaciones web depende de seguir buenas prácticas en cada capa: inputs, base de datos, sesiones, UI y configuración. Una sola vulnerabilidad no corregida puede comprometer toda la aplicación.